



FUERZA AEROSPAIAL
COLOMBIANA



Universidad de
los Andes
Colombia

Departamento de Ingeniería
de Sistemas y Computación

CODEFEST

AD ASTRA 2026

Especificación Técnica

Etapas 1 • Construcción de la Base de Conocimiento

Documento dirigido a los equipos participantes

Versión 1.0 | Julio 2026

Índice

1. Introducción	3
1.1. Contexto del reto	3
1.2. Objetivo de la Etapa 1	4
1.3. Insumos provistos por ADL	4
1.4. Entregables	5
2. Preprocesamiento de las Fuentes	6
2.1. Extracción de texto	6
2.2. Limpieza y normalización	7
2.3. Identificación de documentos	7
3. Chunking: Fragmentación del Texto	7
3.1. ¿Qué es el chunking?	8
3.2. Estrategias de chunking	8
3.3. Requisito de completitud lingüística	9
3.4. Metadata obligatoria por fragmento	9
4. Codificación Semántica: Encoders y Embeddings	9
4.1. ¿Qué son los embeddings?	10
4.2. Encoders vs. decoders	10
4.3. Criterios de selección del encoder	10
4.4. Uso de múltiples encoders	11
5. Gestión del Índice Vectorial con FAISS	11
5.1. ¿Qué es FAISS?	11
5.2. Tipos de índice	12
5.3. Relación entre el índice FAISS y el almacén de metadata	12
5.4. Persistencia del índice	12
6. Flujo Completo de Indexación	13
6.1. Descripción paso a paso	13
7. Grafo de Conocimiento (Componente Bonus)	14
7.1. ¿Qué es un grafo de conocimiento?	14
7.2. Construcción del grafo	14
7.3. Vinculación con la base vectorial	15
8. Módulo de Recuperación	15
8.1. Flujo general de recuperación	15
8.2. Similitud coseno	15
8.3. Restricción sobre modelos generativos	16
8.4. Combinación de múltiples bases vectoriales	16
8.5. Integración con el grafo de conocimiento	17
8.6. Agregación de fragmentos al nivel de documento	17
8.7. Post-filtros basados en vectores o metadata	18

9. Formato de Salida	18
9.1. Diferencia entre fragmento y documento	18
9.2. Estructura de la respuesta por consulta	18
9.2.1. Reglas para cumplir el límite de 250 palabras por fragmento .	18
9.3. Formato del archivo de resultados: JSON Lines	19
9.3.1. Esquema de cada objeto JSON	19
9.3.2. Campos del esquema	20
10. Evaluación	20
10.1. Dataset de evaluación	20
10.2. Métricas de evaluación	20
10.2.1. NDCG@10 para fragmentos	21
10.2.2. F1@3 para documentos	21
10.3. Instrucciones de entrega	22
11. Clasificación de Equipos	22
11.1. Tablas clasificatorias por métrica	22
11.2. Leaderboard unificado mediante Conteo de Borda	22
11.2.1. Procedimiento	23
11.2.2. Ejemplo	23

1 Introducción

1.1 Contexto del reto

En esta versión del CODEFEST AD ASTRA se propone el desarrollo de una arquitectura inteligente multiagente orientada al análisis estratégico de fenómenos con impacto en el dominio aéreo y espacial, a partir de información proveniente de fuentes abiertas. Para garantizar una mirada integral, el análisis deberá construirse desde tres niveles complementarios: una perspectiva global, que permita identificar tendencias emergentes en ámbitos como tecnología espacial, cambio climático, innovación y riesgos globales; una perspectiva regional, que permita comprender dinámicas territoriales relevantes en América Latina y el Caribe; y una perspectiva nacional, fundamentada en información proveniente de observatorios académicos e institucionales colombianos. Los equipos utilizarán como base, fuentes abiertas provenientes de observatorios y centros de pensamiento colombianos, regionales y globales enfocados en temáticas relacionadas con derechos humanos, seguridad, ciencia y tecnología, juventud, empleo, medio ambiente, políticas públicas, desarrollo humano, educación, medios y dinámicas socioeconómicas, entre otros. Estas fuentes permitirán analizar cómo los fenómenos globales y regionales se manifiestan de manera concreta en el contexto colombiano.

- Fenómeno 1.** Inteligencia artificial e innovación en entornos militares: el primer fenómeno de análisis se centra en el estudio del uso, desarrollo e impacto de la inteligencia artificial en la Defensa Nacional a partir de información proveniente de fuentes abiertas y entornos militares. El análisis deberá considerar tendencias globales y regionales en la adopción de IA en el Sector Defensa mediante el uso de observatorios relacionados con ciencia, tecnología, innovación, políticas públicas, educación, empleo y desarrollo de talento. Lo anterior, con el fin de identificar oportunidades, brechas, riesgos y desafíos para el fortalecimiento de capacidades estratégicas mediante IA.
- Fenómeno 2.** Seguridad espacial y órbita baja terrestre: el segundo fenómeno se enfoca en el análisis de la seguridad del entorno espacial, con énfasis en la congestión de la órbita baja terrestre (Low Earth Orbit, LEO) y en los riesgos asociados a la proliferación de objetos espaciales artificiales no funcionales (space debris), tales como satélites obsoletos, fragmentos producto de colisiones, restos de etapas de cohetes y otros elementos orbitales que incrementan la probabilidad de colisiones y afectan la sostenibilidad del uso del espacio. En este reto, los participantes no trabajarán directamente con datos orbitales primarios, sino con productos analíticos provenientes de observatorios, agencias espaciales, organismos multilaterales, centros de pensamiento y estudios académicos especializados en sostenibilidad y seguridad espacial. A partir de estos informes, reportes técnicos y análisis estratégicos, se espera que los equipos identifiquen tendencias, correlaciones y escenarios prospectivos relacionados con la evolución del entorno orbital.

Fenómeno 3. Dinámicas territoriales en América Latina: el tercer fenómeno se orienta al análisis de dinámicas territoriales relevantes en América Latina y el Caribe que puedan tener implicaciones estratégicas para la seguridad, la estabilidad y el desarrollo de los Estados. El análisis deberá considerar variables sociales, políticas, económicas, ambientales y de seguridad humana, incluyendo aspectos como conflicto, gobernanza, desigualdad, migración, violencia, desarrollo humano, juventud, educación, políticas públicas y percepción social. Para ello, los equipos utilizarán documentos de fuentes abiertas regionales e internacionales y contrastarlas con información proveniente de observatorios académicos e institucionales colombianos, con el propósito de identificar patrones, tendencias, riesgos emergentes y oportunidades estratégicas para el país y la región.

La selección de estos fenómenos permite abordar conjuntos documentales de distinta naturaleza, complejidad y estructura, planteando a los participantes el desafío de organizar, procesar y recuperar información de manera eficiente. La competencia se desarrollará en dos etapas: una virtual y una presencial. El presente documento establece los requerimientos técnicos correspondientes a la Etapa 1, la cual constituye la fase virtual del reto y se orienta a la construcción y evaluación de una base de conocimiento vectorial.

La competencia se desarrolla en dos etapas: una virtual y una presencial. Este documento especifica los requerimientos técnicos de la **Etapa 1**, que corresponde a la fase virtual del reto.

1.2 Objetivo de la Etapa 1

El objetivo de esta etapa es construir una **base de conocimiento vectorial** a partir de las fuentes documentales provistas por el equipo organizador. Esta base servirá como infraestructura de recuperación de información para la Etapa 2 (presencial), en la cual se integrará un asistente conversacional.

La Etapa 1 evalúa exclusivamente la **calidad de recuperación** de la base construida: cuán relevantes son los documentos y fragmentos que el sistema retorna ante consultas en lenguaje natural.

1.3 Insumos provistos por ADL

El equipo de ADL descargará y pondrá a disposición de los participantes el contenido de las fuentes documentales seleccionadas para cada fenómeno. Dicho contenido se entregará en los siguientes formatos, sin procesamiento adicional:

- **PDF:** publicaciones técnicas, informes y documentos institucionales.
- **HTML:** páginas web y artículos en línea tal como son servidos por sus fuentes.
- **JSON:** artículos y páginas web estructurados
- **CSV:** datasets de investigación y tablas georreferenciada
- **XLSX:** datasets del AI Index
- **Imágenes:** portadas y algunas figuras de los reportes

- **PBF**: mapas

Cada archivo constituye un **documento** en el sentido del reto y tiene un identificador único. El conjunto de documentos provisto por ADL para los tres fenómenos forma el corpus sobre el cual los equipos deben construir su base de conocimiento.

1.4 Entregables

Al finalizar la Etapa 1, los equipos deberán entregar los siguientes cuatro componentes empaquetados en un único directorio con la estructura indicada a continuación.

1. **Base vectorial** (obligatorio): el índice FAISS y el almacén de metadata correspondiente, organizados en subcarpetas por encoder. Si se implementó más de un encoder, cada uno tiene su propia subcarpeta. El formato estándar requerido es:
 - **index.faiss**: archivo binario del índice FAISS, serializado con la función estándar `faiss.write_index()`. Este archivo es directamente cargable con `faiss.read_index()` sin dependencias adicionales.
 - **metadata.jsonl**: almacén de metadata en formato JSON Lines, con exactamente un objeto JSON por línea. Cada objeto corresponde a un fragmento e incluye todos los campos obligatorios de la Tabla 1. El orden de las líneas debe coincidir con los identificadores internos asignados por FAISS al momento de la indexación.

Si el equipo construyó el grafo de conocimiento opcional (Sección 7), debe incluirse en una subcarpeta **grafo/** como un archivo **grafo.graphml**, formato GraphML, exportable directamente desde NetworkX, Neo4j o cualquier herramienta compatible.

2. **Archivo de resultados** en formato JSON Lines (véase la Sección 9), denominado **resultados.jsonl**, con exactamente 50 líneas correspondientes a las consultas q001–q050.
3. **Documento técnico** en PDF (máximo 8 páginas) que describa las decisiones de diseño tomadas: estrategia de chunking y su justificación, encoder(s) seleccionado(s) y criterios de elección, tipo de índice FAISS empleado, y, de aplicar, la descripción del grafo de conocimiento.
4. **Script Python** que utilice el índice, lea el archivo de consultas y genere el archivo de resultados **resultados.jsonl**. Nombre **generador.py**. El objetivo es reproducir los resultados. Si no es posible reproducir los resultados, se excluirá de la evaluación.

La estructura de directorios esperada es la siguiente:

```
entrega/  
  resultados.jsonl  
  generador.py  
  informe_tecnico.pdf  
  base_vectorial/  
    encoder_<nombre>/  
      index.faiss  
      metadata.jsonl  
    encoder_<nombre2>/ (si aplica)  
      index.faiss  
      metadata.jsonl  
  grafo/ (bonus, si aplica)  
    grafo.graphml
```

2 Preprocesamiento de las Fuentes

2.1 Extracción de texto

Antes de fragmentar y codificar los documentos, es necesario extraer el contenido textual a partir de su formato original. El proceso varía según el tipo de archivo:

- **PDF:** se extrae el texto contenido en cada página mediante herramientas de análisis de documentos PDF. Se recomienda preservar el orden de lectura de los párrafos. El contenido de imágenes, tablas no textuales y elementos decorativos puede omitirse si no aporta significado semántico legible.
- **HTML:** se elimina el marcado (etiquetas, atributos, scripts, estilos) y se conserva únicamente el texto visible. Los elementos estructurales del HTML (encabezados, párrafos, listas) pueden usarse como señales para la estrategia de chunking.
- **Markdown/TXT:** el contenido es mayoritariamente texto plano con marcado ligero. Los encabezados (#, ##) y separadores son especialmente útiles como señales de segmentación.
- **JSON:** dado que el texto ya viene estructurado, se recomienda interpretar el objeto y seleccionar explícitamente los campos que contienen el texto de cada artículo (por ejemplo title, body_text, body_paragraphs), concatenándolos respetando su orden de aparición; si el cuerpo viene como una lista de párrafos, se unen preservando ese orden. Los campos descriptivos (url, date, authors, tags) conviene conservarlos como metadata del documento en lugar de mezclarlos con el cuerpo del texto.
- **CSV/XLSX:** se recomienda leer primero la fila de cabecera y luego recorrer los registros uno a uno, obteniendo cada fila como una secuencia de pares columna:

valor separados por un delimitador, de modo que cada valor conserve el nombre de su columna como contexto. Las celdas vacías pueden omitirse o limpiarse. Cada fila puede tratarse como una unidad de fragmentación independiente.

- **Imágenes:** Cuando la imagen contiene texto relevante (infografías o gráficos), se recomienda aplicar OCR sobre cada imagen para recuperar y verificar el texto para su análisis.
- **PBF:** para extraer su contenido se usa una herramienta que lea ese formato y lo decodifique. Una vez abierto, se recorren las capas y, dentro de cada una, los elementos del mapa (municipios, zonas), leyendo los atributos de cada uno. Esos atributos se pasan a texto como pares atributo: valor. Como el mismo elemento se repite en varios niveles de zoom por cada archivo, es conveniente quedarse con una sola versión para no duplicar la data.

2.2 Limpieza y normalización

Una vez extraído el texto, se aplica una limpieza básica que asegura la calidad del corpus:

- Eliminación de caracteres de control y espacios redundantes.
- Normalización de codificación de caracteres (UTF-8).
- Detección y marcado del idioma predominante del documento.
- Eliminación de elementos repetitivos sin valor informativo (cabeceras de página, pies de página, numeración de páginas, boilerplate de sitios web).

2.3 Identificación de documentos

Cada documento recibe un identificador único (`doc_id`) que lo vincula de forma permanente con su texto original y su fuente. Este identificador es la unidad de referencia para la evaluación a nivel de documento (Sección 10.2.2).

Documento

En el contexto de este reto, un **documento** corresponde a un archivo individual provisto por ADL (un PDF, un HTML, un TXT, un CSV, u otro formato de texto). Cada documento tiene un `doc_id` único e inmutable asignado por el equipo participante.

3 Chunking: Fragmentación del Texto

3.1 ¿Qué es el chunking?

Chunking

El **chunking** (fragmentación) es el proceso de dividir un documento de texto en unidades más pequeñas denominadas **fragmentos** o **chunks**. Estas unidades son las que se codifican individualmente como vectores y se almacenan en la base de conocimiento.

El chunking es necesario porque los modelos de codificación tienen un límite en la cantidad de texto que pueden procesar a la vez, y porque los documentos completos son demasiado heterogéneos en contenido como para ser representados por un único vector. Un fragmento bien delimitado contiene una idea o argumento cohesivo, lo que permite que su vector capture con precisión su significado semántico.

3.2 Estrategias de chunking

Existen varias estrategias de fragmentación, con distintas características:

Por tamaño fijo de tokens Se divide el texto cada n tokens o palabras, sin considerar la estructura lingüística. Es simple de implementar pero puede cortar oraciones o párrafos en puntos arbitrarios.

Por oración Se utilizan los límites de oraciones (puntos, signos de interrogación, signos de exclamación) como fronteras naturales de corte. Produce fragmentos cortos pero lingüísticamente completos.

Por párrafo Se respetan los saltos de párrafo del texto original como unidades de segmentación. Es la estrategia más cercana a la estructura semántica del autor.

Jerárquica o estructural Se aprovechan las señales estructurales del documento (encabezados en Markdown o HTML, secciones en PDF) para delimitar fragmentos de tamaño variable pero temáticamente cohesivos.

Semántica con superposición Se combinan fragmentos con una ventana deslizante que permite solapamiento entre chunks consecutivos. Esto reduce el riesgo de que una idea relevante quede fragmentada entre dos chunks sin solapamiento.

Los equipos pueden diseñar estrategias híbridas que combinen elementos de las anteriores. Lo que se exige es que la estrategia elegida se justifique explícitamente en el documento técnico entregable.

3.3 Requisito de completitud lingüística

Requisito obligatorio

Ningún fragmento puede contener oraciones o frases incompletas. Los cortes entre chunks consecutivos deben realizarse únicamente en límites oracionales completos. Una oración que comienza en un chunk debe terminar en ese mismo chunk; no se permite que una oración se extienda a través de la frontera entre dos fragmentos.

Este requisito tiene implicaciones directas sobre estrategias de tamaño fijo: si se fija un tamaño máximo de n tokens, el corte efectivo debe retroceder al final de la última oración completa que quepa dentro de ese límite.

3.4 Metadata obligatoria por fragmento

Cada chunk almacenado en la base vectorial debe tener asociados los siguientes campos de metadata. Esta metadata se utiliza en la recuperación, en la construcción de la respuesta y en la trazabilidad de los resultados.

Tabla 1: Campos de metadata obligatorios por fragmento.

Campo	Tipo	Descripción
doc_id	cadena	Identificador único del documento de origen.
chunk_id	cadena	Identificador único del fragmento dentro del documento.
fuelle	cadena	Nombre o URL del archivo original provisto por ADL.
formato	cadena	Formato del archivo de origen: pdf, html o md.
fenomeno	entero	Fenómeno temático al que pertenece el documento (1, 2 o 3).
posicion	entero	Índice ordinal del fragmento dentro del documento (empieza en 0).
num_tokens	entero	Número de tokens del fragmento.
texto	cadena	Texto original del fragmento, sin modificaciones.

Los equipos pueden añadir campos adicionales (idioma, fecha de publicación, título del documento, etc.) siempre que los campos obligatorios de la Tabla 1 estén presentes.

4 Codificación Semántica: Encoders y Embeddings

4.1 ¿Qué son los embeddings?

Embedding

Un **embedding** es una representación numérica densa de un texto en un espacio vectorial de dimensión fija $d \in \mathbb{N}$. Formalmente, dado un fragmento de texto t , un encoder ϕ produce:

$$\mathbf{v} = \phi(t) \in \mathbb{R}^d \quad (1)$$

donde $\mathbf{v}$ es el vector que captura el significado semántico de t . Textos con contenido similar producen vectores próximos en el espacio, medidos por una métrica de similitud apropiada.

4.2 Encoders vs. decoders

Los modelos de lenguaje basados en transformers se pueden clasificar en dos familias según su arquitectura:

- **Encoders** (familia BERT): procesan el texto de forma bidireccional y producen representaciones contextualizadas de cada token. Son la herramienta natural para generar embeddings de fragmentos, ya que su objetivo de entrenamiento consiste en comprender el contexto completo de un texto de entrada.
- **Decoders** (familia GPT): generan texto de forma autoregresiva, prediciendo el siguiente token a partir de los anteriores. No son el instrumento adecuado para generar embeddings de recuperación, y su uso está explícitamente **prohibido** en las etapas de construcción del índice y de recuperación de esta Etapa 1 (véase Sección 8.3).

Los equipos deben utilizar modelos encoder disponibles públicamente en HuggingFace bajo licencias de uso libre.

4.3 Criterios de selección del encoder

La elección del encoder o encoders tiene un impacto directo en la calidad de la recuperación. Los equipos deben justificar su selección en función de los siguientes criterios:

Soporte multilingüe El corpus del reto contiene documentos en español, inglés y portugués. Un encoder que opere de forma nativa en los tres idiomas producirá representaciones comparables entre documentos de distintos idiomas, lo cual es esencial para que una consulta en español pueda recuperar documentos en inglés o portugués.

Dimensionalidad del vector La dimensión d del vector de salida afecta directamente el costo de almacenamiento, la velocidad de búsqueda y, en cierta medida, la expresividad de la representación. Dimensiones más altas no garantizan mejor rendimiento.

Longitud máxima de entrada Los modelos encoder tienen un límite de tokens de entrada (comunmente 512 tokens). Este límite condiciona la estrategia de chunking: los fragmentos deben diseñarse para no superar dicho límite.

Rendimiento en benchmarks Benchmarks públicos como MTEB (*Massive Text Embedding Benchmark*) y BEIR evalúan la calidad de los embeddings en tareas de recuperación de información. Un encoder con buen desempeño en recuperación densa es preferible a uno optimizado para otras tareas (clasificación, similitud de pares).

Licencia El modelo debe tener una licencia que permita su uso en el contexto del reto. Se prefieren licencias Apache 2.0, MIT o CC BY.

Eficiencia computacional Dado que los equipos disponen de recursos de cómputo limitados, el tiempo de inferencia y el consumo de memoria son factores relevantes.

4.4 Uso de múltiples encoders

Los equipos pueden construir su base vectorial con más de un encoder, operando de forma complementaria. Esta estrategia puede ser ventajosa cuando:

- Un encoder tiene mejor cobertura para ciertos idiomas o dominios específicos.
- Se desea combinar representaciones de granularidad diferente (e.g., un encoder de alta dimensionalidad para precisión semántica y uno de baja dimensionalidad para eficiencia).
- Se busca mejorar la robustez de la recuperación combinando rankings producidos por diferentes espacios vectoriales.

Si se emplean múltiples encoders, cada uno genera su propio índice FAISS independiente. La forma de combinar los resultados de estos índices en la etapa de recuperación se describe en la Sección 8.

5 Gestión del Índice Vectorial con FAISS

5.1 ¿Qué es FAISS?

FAISS

FAISS (*Facebook AI Similarity Search*) es una biblioteca de código abierto desarrollada por Meta AI Research para la búsqueda eficiente de vecinos más cercanos en espacios vectoriales de alta dimensión. Está escrita en C++ con interfaces para Python, y está optimizada para funcionar tanto en CPU como en GPU.

FAISS permite indexar millones de vectores y realizar búsquedas de similitud en tiempos del orden de milisegundos. En el contexto de este reto, FAISS es la herramienta obligatoria para gestionar los vectores generados por los encoders.

5.2 Tipos de índice

FAISS ofrece varios tipos de índice con distintas compensaciones entre exactitud, velocidad y uso de memoria:

IndexFlatL2 / IndexFlatIP Índices planos que realizan búsqueda exacta por distancia L2 (euclidiana) o por producto interno (*inner product*), respectivamente. Garantizan el resultado exacto pero no escalan bien a conjuntos de datos muy grandes, pues comparan el vector de consulta contra todos los vectores del índice.

IndexIVFFlat Índice basado en cuantificación vectorial invertida (*Inverted File Index*). El espacio vectorial se divide en k celdas mediante k -medias. En la búsqueda, solo se exploran las n_{probe} celdas más cercanas a la consulta, lo que reduce el costo computacional a cambio de una pequeña pérdida en exactitud.

IndexHNSW Índice basado en grafos de mundo pequeño jerárquico (*Hierarchical Navigable Small World*). Ofrece búsqueda aproximada de alta calidad con tiempos de consulta muy bajos, a cambio de un mayor uso de memoria.

Para el volumen de documentos esperado en este reto, un índice plano (**IndexFlatIP** con vectores normalizados, equivalente a similitud coseno) es suficiente y garantiza resultados exactos. Los índices aproximados son apropiados si el equipo trabaja con corpus de gran escala o si el tiempo de respuesta es un factor crítico.

5.3 Relación entre el índice FAISS y el almacén de metadata

FAISS almacena únicamente los vectores numéricos y sus identificadores enteros internos. La metadata asociada a cada fragmento (Tabla 1) debe mantenerse en un almacén separado que mapee el identificador interno de FAISS al `chunk_id` y al resto de los campos.

Esta separación implica que el sistema de recuperación tiene dos componentes:

1. **Índice FAISS:** recibe un vector de consulta y devuelve los k identificadores internos más similares junto con sus puntuaciones.
2. **Almacén de metadata:** dado un identificador interno, devuelve el texto del fragmento, el `doc_id` de origen y el resto de la metadata.

5.4 Persistencia del índice

El índice FAISS debe guardarse en disco de forma que pueda cargarse sin necesidad de reindexar todo el corpus. FAISS proporciona funciones nativas para serializar y deserializar índices. El almacén de metadata puede persistirse en cualquier formato estructurado (JSON, CSV, base de datos SQLite, etc.) siempre que la correspondencia entre identificadores FAISS y `chunk_id` sea exacta y reproducible.

6 Flujo Completo de Indexación

La Figura 1 presenta el flujo de extremo a extremo para construir la base de conocimiento vectorial. Las secciones anteriores detallan cada etapa; esta sección describe la integración del proceso completo.

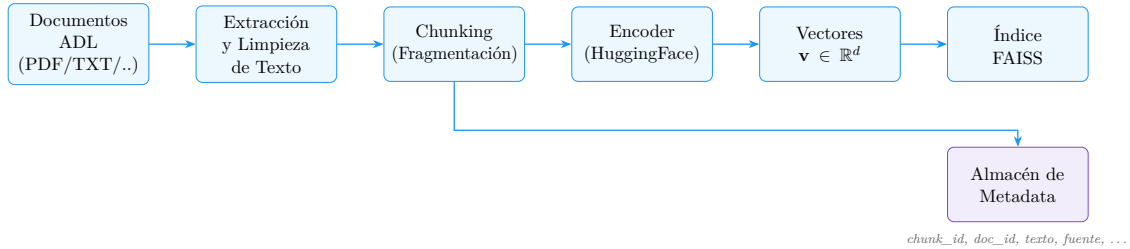


Figura 1: Flujo completo de indexación: de los documentos provisto por ADL al índice FAISS.

6.1 Descripción paso a paso

1. **Documentos ADL:** los archivos en PDF, HTML y Markdown constituyen la entrada del sistema. Cada archivo es un documento con un `doc_id` asignado.
2. **Extracción y limpieza:** se extrae el texto plano de cada documento y se aplica la normalización descrita en la Sección 2. El resultado es un texto limpio listo para ser fragmentado.
3. **Chunking:** el texto limpio se divide en fragmentos según la estrategia definida por el equipo (Sección 3). A cada fragmento se le asigna su `chunk_id` y se construye su registro de metadata.
4. **Encoder:** cada fragmento se pasa al modelo encoder. El encoder produce un vector $\mathbf{v} \in \mathbb{R}^d$ que representa semánticamente el contenido del fragmento. Si se emplean múltiples encoders, este paso se repite para cada uno, generando un conjunto de vectores por fragmento.
5. **Vectores:** el conjunto de vectores de todos los fragmentos del corpus está listo para ser indexado. Si los vectores serán comparados mediante similitud coseno, deben normalizarse a norma unitaria antes de insertarlos en el índice.
6. **Índice FAISS:** los vectores normalizados se insertan en el índice FAISS. El índice asigna un identificador interno entero a cada vector. Este identificador debe vincularse al `chunk_id` correspondiente en el almacén de metadata.
7. **Almacén de metadata:** en paralelo, el registro de metadata de cada fragmento se persiste en el almacén externo. La clave de acceso es el identificador interno FAISS o el `chunk_id`.

7 Grafo de Conocimiento (Componente Bonus)

Esta sección describe un componente opcional que otorga puntuación adicional en la evaluación. Su implementación no es obligatoria para participar en el reto.

7.1 ¿Qué es un grafo de conocimiento?

Grafo de conocimiento

Un **grafo de conocimiento** (*knowledge graph*) es una representación estructurada del conocimiento en forma de grafo dirigido, donde los nodos representan **entidades** (personas, organizaciones, lugares, conceptos, eventos) y las aristas representan **relaciones** tipadas entre dichas entidades. Formalmente:

$$G = (E, R, T), \quad T \subseteq E \times R \times E \quad (2)$$

donde E es el conjunto de entidades, R el conjunto de tipos de relaciones, y T el conjunto de tripletas $(e_{\text{sujeto}}, r, e_{\text{objeto}})$ que describen hechos del dominio.

Por ejemplo, para el fenómeno 1 del reto podrían existir tripletas como:

(EE.UU., desarrolla, sistema-de-armas-autónomo)
(sistema-de-armas-autónomo, regula, convenio-ginebra)

El grafo de conocimiento complementa la base vectorial porque captura **relaciones explícitas** entre entidades que los embeddings solo representan de forma implícita a través de la proximidad semántica.

7.2 Construcción del grafo

La construcción del grafo a partir del corpus del reto sigue tres etapas:

1. **Reconocimiento de entidades (NER)**: se identifican y clasifican las entidades mencionadas en el texto (personas, organizaciones, países, tecnologías, eventos, etc.) mediante modelos de reconocimiento de entidades nombradas. Se recomiendan modelos open source disponibles en HuggingFace entrenados para NER multilingüe.
2. **Extracción de relaciones (RE)**: se identifican las relaciones semánticas entre pares de entidades dentro del mismo fragmento o párrafo. Esto puede realizarse mediante modelos de clasificación de relaciones, heurísticas basadas en patrones lingüísticos, o dependencias sintácticas.
3. **Construcción e integración del grafo**: las tripletas extraídas se almacenan en una estructura de grafo. Cada tripleta mantiene una referencia al `doc_id` y al `chunk_id` de origen, lo que permite rastrear la evidencia textual de cada relación. Herramientas como **NetworkX** (Python, en memoria), **Neo4j** (base de datos de grafos) o **RDFLib** (grafos RDF) pueden emplearse para este fin.

7.3 Vinculación con la base vectorial

Cada entidad del grafo puede vincularse con los chunks del índice FAISS que la mencionan. Esto permite que, en la etapa de recuperación, el grafo provea caminos de relaciones como evidencia complementaria a la similitud vectorial. La integración se describe en la Sección 8.5.

8 Módulo de Recuperación

8.1 Flujo general de recuperación

Dado una consulta del usuario expresada en lenguaje natural, el módulo de recuperación sigue los pasos ilustrados en la Figura 2:

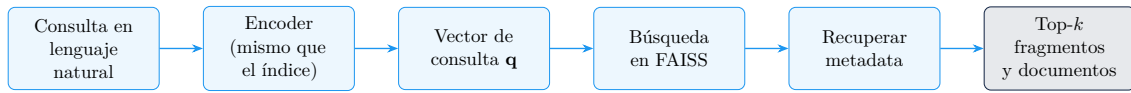


Figura 2: Flujo de recuperación: de la consulta al resultado.

1. La consulta del usuario se pasa al mismo encoder utilizado durante la indexación. Es imprescindible utilizar el **mismo encoder** para garantizar que los vectores de consulta y los vectores del índice habiten el mismo espacio semántico.
2. El encoder produce el vector de consulta $\mathbf{q} \in \mathbb{R}^d$. Si los vectores del índice están normalizados, $\mathbf{q}$ debe normalizarse también.
3. FAISS compara $\mathbf{q}$ contra todos los vectores del índice y devuelve los k fragmentos más similares con sus puntuaciones.
4. Para cada identificador devuelto por FAISS se recupera la metadata correspondiente del almacén externo, incluyendo el texto del fragmento y el `doc_id` de origen.
5. A partir de los chunks recuperados se construyen los dos niveles de resultado requeridos: fragmentos (nivel chunk) y documentos (nivel `doc_id`).

8.2 Similitud coseno

La medida de similitud empleada para comparar el vector de consulta con los vectores del índice es la **similitud coseno**, definida como:

$$\cos(\mathbf{q}, \mathbf{v}) = \frac{\mathbf{q} \cdot \mathbf{v}}{\|\mathbf{q}\| \|\mathbf{v}\|} \quad (3)$$

donde $\mathbf{q} \cdot \mathbf{v} = \sum_{i=1}^d q_i v_i$ es el producto punto, y $\|\mathbf{u}\| = \sqrt{\sum_{i=1}^d u_i^2}$ es la norma euclidiana del vector $\mathbf{u}$.

El valor de la similitud coseno está en el rango $[-1, 1]$. Cuando ambos vectores apuntan en la misma dirección ($\cos = 1$) el significado es idéntico; cuando son

ortogonales ($\cos = 0$) no tienen relación semántica; cuando apuntan en direcciones opuestas ($\cos = -1$) el significado es antagónico.

Si los vectores están normalizados a norma unitaria ($\|\mathbf{q}\| = \|\mathbf{v}\| = 1$), la similitud coseno es equivalente al producto interno:

$$\cos(\mathbf{q}, \mathbf{v}) = \mathbf{q} \cdot \mathbf{v} \quad (\text{para vectores normalizados}) \quad (4)$$

Esto es aprovechado por `IndexFlatIP` de FAISS, que maximiza el producto interno y por tanto equivale a maximizar la similitud coseno cuando los vectores están normalizados.

8.3 Restricción sobre modelos generativos

En ninguna etapa del proceso de recuperación se permite el uso de modelos de lenguaje generativos (arquitecturas decoder, como GPT, LLaMA, Gemini, Claude, o similares). Esto incluye, sin limitarse a:

- reordenamiento (*reranking*) de resultados mediante un LLM;
- filtrado o selección de fragmentos basado en la generación de texto;
- reformulación o expansión de la consulta mediante un decoder;
- síntesis o resumen de fragmentos para construir el resultado.

La recuperación debe operar exclusivamente sobre vectores, puntuaciones de similitud y metadata. Se permiten post-filtros que operen directamente sobre los vectores recuperados o sobre los campos de metadata (e.g., filtrar por fenómeno, por idioma, por rango de fechas).

8.4 Combinación de múltiples bases vectoriales

Si el equipo construyó índices FAISS con más de un encoder, cada índice produce de forma independiente una lista ordenada de fragmentos para la misma consulta. Estas listas deben combinarse en una sola lista unificada sin recurrir a modelos generativos. Las estrategias más comunes son:

CombSUM Para cada fragmento, se suman las puntuaciones de similitud coseno obtenidas en cada índice. Si un fragmento no aparece en el top- k de algún índice, se le asigna puntuación cero en ese índice.

$$s_{\text{CombSUM}}(c) = \sum_{j=1}^m s_j(c) \quad (5)$$

donde $s_j(c)$ es la puntuación del fragmento c en el índice j y m es el número de encoders.

CombMNZ Similar a CombSUM, pero multiplica la suma por el número de índices en los que el fragmento aparece, favoreciendo la consistencia entre encoders:

$$s_{\text{CombMNZ}}(c) = \left(\sum_{j=1}^m s_j(c) \right) \cdot |\{j : s_j(c) > 0\}| \quad (6)$$

Reciprocal Rank Fusion (RRF) En lugar de combinar puntuaciones, combina los rangos $r_j(c)$ de cada fragmento en cada índice:

$$s_{\text{RRF}}(c) = \sum_{j=1}^m \frac{1}{k_0 + r_j(c)} \quad (7)$$

donde k_0 es una constante de suavizado (típicamente $k_0 = 60$) y $r_j(c)$ es la posición del fragmento c en la lista ordenada del índice j (empieza en 1). RRF es robusto a la diferencia de escalas entre puntuaciones de distintos encoders.

8.5 Integración con el grafo de conocimiento

Si el equipo implementó el grafo de conocimiento opcional (Sección 7), puede combinarlo con los resultados vectoriales de la siguiente manera:

1. Se identifican las entidades mencionadas en la consulta mediante el mismo componente NER utilizado durante la construcción del grafo.
2. Se consulta el grafo para recuperar los chunks vinculados a dichas entidades y a las entidades directamente relacionadas con ellas (vecinos de primer orden en el grafo).
3. Los chunks recuperados por el grafo se incorporan al pool de candidatos y se les asigna una puntuación basada en el número de relaciones relevantes encontradas.
4. Este pool de candidatos del grafo se fusiona con los resultados vectoriales de FAISS mediante una de las estrategias de combinación descritas en la Sección 8.4 (e.g., RRF tratando el grafo como un índice adicional).

8.6 Agregación de fragmentos al nivel de documento

La evaluación requiere tanto resultados a nivel de **fragmento** como a nivel de **documento**. La agregación de chunks al nivel de documento se realiza así:

1. Los k_{chunk} fragmentos más relevantes (con sus puntuaciones) se agrupan por `doc_id`.
2. Para cada documento, se calcula una puntuación agregada. Las estrategias habituales son: puntuación del fragmento más relevante del documento (*max pooling*), suma de las puntuaciones de todos sus fragmentos recuperados, o media ponderada.
3. Los documentos se ordenan de mayor a menor puntuación agregada y se seleccionan los 3 más relevantes.

Este proceso opera exclusivamente sobre las puntuaciones numéricas producidas por FAISS, sin intervención de modelos generativos.

8.7 Post-filtros basados en vectores o metadata

Los equipos pueden aplicar filtros adicionales sobre los resultados de FAISS antes de construir la respuesta final. Estos filtros deben operar directamente sobre:

- **Metadata:** filtrar por campo `fenomeno`, `formato`, rango de fechas, idioma, etc.
- **Vectores:** por ejemplo, descartar fragmentos cuya similitud coseno con la consulta sea inferior a un umbral mínimo θ .

9 Formato de Salida

9.1 Diferencia entre fragmento y documento

Fragmento vs. documento

Un **fragmento** (*chunk*) es la unidad mínima de texto que se almacena en la base vectorial, identificado por su `chunk_id`. Corresponde a una porción del documento original delimitada por la estrategia de chunking del equipo.

Un **documento** es el archivo original provisto por ADL, identificado por su `doc_id`. Un documento contiene uno o más fragmentos. La relevancia de un documento se infiere a partir de la relevancia de sus fragmentos (véase Sección 8.6).

Esta distinción es importante para la evaluación. El sistema devuelve resultados en ambos niveles de granularidad y cada nivel se evalúa con una métrica diferente.

9.2 Estructura de la respuesta por consulta

Para cada consulta del conjunto de evaluación, el sistema debe devolver:

1. **Los 3 documentos más relevantes:** lista ordenada de 3 `doc_id`, de mayor a menor relevancia agregada.
2. **Los 10 fragmentos más relevantes:** lista ordenada de 10 objetos, cada uno con el texto del fragmento y su `chunk_id`. Cada fragmento debe tener **como máximo 250 palabras**.

9.2.1 Reglas para cumplir el límite de 250 palabras por fragmento

- Si un chunk recuperado tiene **más de 250 palabras**, debe dividirse en sub-fragmentos que respeten el límite. La división debe respetar el requisito de completitud lingüística (sin oraciones cortadas).
- Si un chunk recuperado tiene **menos de 250 palabras** y se desea combinarlo con fragmentos adyacentes del mismo documento para enriquecer el contexto, puede concatenarse con el fragmento inmediatamente anterior o posterior, siempre que el resultado no supere las 250 palabras.

- Si se realizan divisiones o concatenaciones, el `chunk_id` reportado debe ser el del fragmento original del índice que originó la selección. En consecuencia, cuando un mismo chunk se divide en varios sub-fragmentos, todos ellos comparten el mismo `chunk_id`; esto es aceptable, ya que la relevancia se evalúa sobre el campo `text` y el `chunk_id` cumple aquí una función de trazabilidad, no de emparejamiento (véase la Sección 10.2.1). Cada sub-fragmento debe ocupar su propia posición (`rank`) en la lista de 10 fragmentos.

9.3 Formato del archivo de resultados: JSON Lines

Los resultados se entregan en un único archivo con extensión `.jsonl` (*JSON Lines*). En este formato, **cada línea del archivo es un objeto JSON válido e independiente**, correspondiente a una consulta. No existe ningún separador adicional entre líneas; los saltos de línea son los únicos delimitadores.

El archivo completo tendrá exactamente **50 líneas**, una por cada consulta del conjunto de evaluación.

9.3.1 Esquema de cada objeto JSON

```
{
  "query_id": "q001",
  "documents": [
    {"rank": 1, "doc_id": "DOC-042"},
    {"rank": 2, "doc_id": "DOC-017"},
    {"rank": 3, "doc_id": "DOC-091"}
  ],
  "fragments": [
    {
      "rank": 1,
      "chunk_id": "DOC-042-chunk-007",
      "doc_id": "DOC-042",
      "text": "Texto del fragmento de hasta 250 palabras
        ..."
    },
    {
      "rank": 2,
      "chunk_id": "DOC-017-chunk-003",
      "doc_id": "DOC-017",
      "text": "Texto del fragmento de hasta 250 palabras
        ..."
    }
  ]
}
```

Tabla 2: Campos del objeto JSON de resultado por consulta.

Campo	Tipo	Descripción
<code>query_id</code>	cadena	Identificador de la consulta (q001–q050).
<code>documents</code>	array	Lista de exactamente 3 objetos de documento, ordenados de mayor a menor relevancia.
<code>documents[i].rank</code>	entero	Posición en el ranking (1, 2, 3).
<code>documents[i].doc_id</code>	cadena	Identificador del documento.
<code>fragments</code>	array	Lista de exactamente 10 objetos de fragmento, ordenados de mayor a menor relevancia.
<code>fragments[i].rank</code>	entero	Posición en el ranking (1 a 10).
<code>fragments[i].chunk_id</code>	cadena	Identificador del chunk recuperado del índice.
<code>fragments[i].doc_id</code>	cadena	Identificador del documento de origen del fragmento.
<code>fragments[i].text</code>	cadena	Texto del fragmento (≤ 250 palabras).

9.3.2 Campos del esquema

Requisito obligatorio

El archivo de resultados debe cumplir estrictamente con el esquema anterior. Objetos con campos faltantes, arrays con un número diferente de elementos (distinto de 3 para documentos y de 10 para fragmentos), o fragmentos que superen las 250 palabras serán penalizados o descartados durante la evaluación automática.

10 Evaluación

10.1 Dataset de evaluación

El conjunto de evaluación está compuesto por **50 consultas en lenguaje natural**, identificadas como q001 hasta q050. Las consultas se distribuyen de forma equilibrada entre los tres fenómenos temáticos del reto y en los tres idiomas del corpus (español, inglés y portugués).

Cada consulta tiene asociada una **lista de relevancia** (*ground truth*) construida manualmente por el equipo de CODEFEST, que indica qué documentos y qué fragmentos son relevantes para esa consulta y en qué grado. Esta lista no es pública durante el reto; su comparación con los resultados entregados por los equipos determina las métricas de evaluación.

10.2 Métricas de evaluación

Se utilizan dos métricas, una por cada nivel de granularidad:

10.2.1 NDCG@10 para fragmentos

El **NDCG** (*Normalized Discounted Cumulative Gain*) mide la calidad de una lista ordenada de resultados, penalizando más los errores en las posiciones superiores del ranking. Para los fragmentos se evalúa con corte en $k = 10$ (NDCG@10).

Dado el vector de relevancia $\mathbf{r} = (r_1, r_2, \dots, r_{10})$, donde $r_i \geq 0$ es la relevancia del fragmento en la posición i de la lista entregada por el equipo:

$$\text{DCG}@k = \sum_{i=1}^k \frac{r_i}{\log_2(i+1)} \quad (8)$$

$$\text{NDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k} \quad (9)$$

donde $\text{IDCG}@k$ es el DCG del ranking ideal (los fragmentos más relevantes en el orden óptimo). $\text{NDCG}@10 \in [0, 1]$, siendo 1 el resultado perfecto.

La puntuación final NDCG@10 reportada para el equipo es la media sobre las 50 consultas:

$$\overline{\text{NDCG}@10} = \frac{1}{50} \sum_{q=1}^{50} \text{NDCG}@10(q) \quad (10)$$

Clave de emparejamiento en la evaluación

La relevancia de cada fragmento se juzga sobre su contenido textual (campo `text`). El `chunk_id` no es la clave de emparejamiento con el *ground truth*: dado que cada equipo aplica su propia estrategia de chunking, sus `chunk_id` son identificadores internos y se conservan únicamente con fines de trazabilidad y verificación. Análogamente, a nivel de documento el emparejamiento con el *ground truth* se realiza a través del campo `fuelle` (archivo original provisto por ADL), no del `doc_id` arbitrario asignado por el equipo.

10.2.2 F1@3 para documentos

El **F1@3** evalúa la calidad del conjunto de 3 documentos devueltos para cada consulta, comparándolo con el conjunto de documentos relevantes en el *ground truth*. Se trata de una métrica de conjunto (no considera el orden de los documentos).

Para una consulta q , se definen:

$$P@3 = \frac{|\hat{D}_q \cap D_q^*|}{|\hat{D}_q|} = \frac{|\hat{D}_q \cap D_q^*|}{3} \quad (11)$$

$$R@3 = \frac{|\hat{D}_q \cap D_q^*|}{\min(|D_q^*|, 3)} \quad (12)$$

$$\text{F1@3} = \frac{2 \cdot P@3 \cdot R@3}{P@3 + R@3} \quad (13)$$

donde $\hat{D}_q$ es el conjunto de 3 `doc_id` devueltos por el equipo para la consulta q , y D_q^* es el conjunto de documentos relevantes según el *ground truth*. El denominador de $R@3$ se limita a $\min(|D_q^*|, 3)$ para no penalizar casos en los que el número de documentos relevantes es inferior a 3.

La puntuación final $F1@3$ del equipo es la media sobre las 50 consultas:

$$\overline{F1@3} = \frac{1}{50} \sum_{q=1}^{50} F1@3(q) \quad (14)$$

10.3 Instrucciones de entrega

Requisito obligatorio

Los equipos deben entregar un único archivo `resultados.jsonl` con exactamente 50 líneas, una por consulta, en el orden `q001`, `q002`, ..., `q050`. El archivo debe cumplir el esquema especificado en la Sección 9. Entregas fuera de plazo o con formato incorrecto no serán consideradas en la evaluación.

11 Clasificación de Equipos

11.1 Tablas clasificatorias por métrica

Se construirán dos tablas clasificatorias independientes:

- **Tabla A — $NDCG@10$:** ordena los equipos de mayor a menor $\overline{NDCG@10}$ (nivel fragmento).
- **Tabla B — $F1@3$:** ordena los equipos de mayor a menor $\overline{F1@3}$ (nivel documento).

En caso de empate en cualquiera de las dos tablas, el desempate se resolverá por la puntuación en la otra métrica. Si el empate persiste, se mantendrán los equipos en la misma posición compartida.

11.2 Leaderboard unificado mediante Conteo de Borda

Para combinar ambas clasificaciones en un único ranking final, se aplica el **Conteo de Borda** (*Borda Count*), un método de votación por posiciones que es robusto ante diferencias de escala entre métricas.

Conteo de Borda

Sea N el número de equipos participantes. En el Conteo de Borda, al equipo que ocupa la posición p en una tabla clasificatoria se le asignan $N - p$ puntos de Borda para esa tabla. El equipo con mejor posición obtiene $N - 1$ puntos y el equipo con peor posición obtiene 0 puntos.

La puntuación de Borda total de cada equipo es la suma de sus puntos en las dos tablas:

$$B_i = B_i^{NDCG} + B_i^{F1} \quad (15)$$

donde B_i^{NDCG} y B_i^{F1} son los puntos de Borda del equipo i en la Tabla A y la Tabla B, respectivamente.

11.2.1 Procedimiento

1. Construir la Tabla A ordenando los equipos por $\overline{\text{NDCG@10}}$. Asignar puntos de Borda: $B_i^{\text{NDCG}} = N - p_i^A$, donde p_i^A es la posición del equipo i en la Tabla A.
2. Construir la Tabla B ordenando los equipos por $\overline{\text{F1@3}}$. Asignar puntos de Borda: $B_i^{\text{F1}} = N - p_i^B$.
3. Calcular $B_i = B_i^{\text{NDCG}} + B_i^{\text{F1}}$ para cada equipo.
4. Ordenar los equipos de mayor a menor B_i . El resultado es el **Leaderboard Unificado**.

En caso de empate en puntos de Borda, el desempate se realiza por $\overline{\text{NDCG@10}}$.

11.2.2 Ejemplo

Tabla 3: Ejemplo de Conteo de Borda con 4 equipos hipotéticos.

Equipo	$\overline{\text{NDCG@10}}$	Pos. A	B^{NDCG}	$\overline{\text{F1@3}}$	Pos. B	B^{F1}	B total
Equipo 1	0.72	2	2	0.68	1	3	5
Equipo 2	0.81	1	3	0.55	3	1	4
Equipo 3	0.65	3	1	0.62	2	2	3
Equipo 4	0.50	4	0	0.41	4	0	0

En este ejemplo, el Equipo 1 lidera el Leaderboard Unificado a pesar de no ocupar el primer lugar en ninguna de las dos tablas individuales, gracias a su desempeño consistente en ambas métricas.